

1 How to Build a Scheduler

Up till now we have used a very simple loop such as

```
while(1)
{
    taskA();
    taskB();
    taskC();
    time_delay();
}
```

New features

- The ability of a task to halt itself.
- The ability of a task to sleep for a while.

1.1 Part I Using function pointers to start a task

We learned that the syntax:

```
type (*name)(type arg1, type arg2, ...)
```

indicates a function pointer called `name` which can point to any function having the same prototype. For example

```
int (*fp)(char x, int y, double z)
```

Example: start function

```
void start_function(void (*functionPTR)() )  
{  
    functionPTR();  
}
```

Now our basic scheduler could be written:

```
while(1)
{
    start_function( taskA);
    start_function( taskB);
    start_function( taskC);
    time_delay();
}
```

1.2 Part II Keeping lists of functions

We can create an array of function pointers as follows:

```
#define  NUMBER_OF_FUNCTIONS      10
```

```
void (*list_of_functions[NUMBER_OF_FUNCTIONS])(void *p)
```

Here then is some pseudocode for a main program and scheduler function:

```
//function prototypes for tasks
```

```
void taskA(void *p);
```

```
void taskB(void *p);
```

```
...
```

```
// function prototype for scheduler
```

```
void scheduler();
```

```
#define NTASKS      4
```

```
//lets make the task list global
```

```
// an array of function pointers, each one has a void* parameter
```

```
void (*readytasks[NTASKS]) (void *p);
```



```
main()
{
    // initialize array of pointers to tasks
    readytasks[0] = taskA;
    readytasks[1] = taskB;
    ...
    readytasks[3]= NULL;  // NULL signals the last task in the list

    // now start scheduler
    while(1) {
        scheduler();
        time_delay();  // tune or use timer for 1ms loop time
    }
} // end of main
```

Scheduler function (pseudocode)

```
scheduler() {  
    if(readytasks[task_index] == NULL && task_index != 0) task_index = 0;  
    if(readytasks[task_index] == NULL && task_index == 0) {  
        // figure out something to do because there are no tasks  
    }  
    start_function(readytasks[task_index]);  
    task_index++; // Round Robin/we're taking turns  
    return;  
}
```

1.3 part III Manipulating the Task Lists

halt_me function

```
halt_me() {  
    * identify which task is currently running (i.e. look at task  
    * copy the function pointer from readytasks[task_index]  
      to the haltedtasks array  
    * move the remaining tasks up in readytasks[] to fill the  
      empty hole and copy NULL into the last element.  
    * increment the index of the haltedtasks array.  
    return;  
}
```

When `halt_me` returns to the task, the task should `return` and then it will not be started again by the scheduler since the current task has been deleted from `readytasks`

Sleep

Typical Task using sleep()

```
task_c(p) {  
    compute for a while;  
    sleep(10);  
    return;  
}
```

Sleep function (pseudocode)

```
sleep(int d) {  
    * copy function pointer from readytasks[task_index]  
      to the waitingtasks[] array.  
    * clean up readytasks[] as in halt_me();  
    * copy the d into the delays array with the same index  
      as the function pointer has in waitingtasks[]  
}
```

Keeping track of sleep delays

```
...
* for each element of waitingtasks which is not NULL:
    decrement the delay value d.
    if (d==0) move the function pointer to the end
               of the readytasks[] array and remove it
               from waitingtasks.
// end of scheduler
return;
}
```

1.4 An Alternate Data Structure for Scheduling

Let's set up a struct which contains everything we need to track about a Task:

```
typedef struct TCBstruct {  
    void (*ftpr)(void *p);    // the function pointer  
    void *arg_ptr;            // the argument pointer  
    unsigned short int  state; // the task state  
    unsigned int delay;       // sleep delay  
} tcb;
```



```
#define STATE_RUNNING    0
#define STATE_READY      1
#define STATE_WAITING    2
#define STATE_INACTIVE   3
```

```
tcb TaskList[N_MAX_TASKS];
```

Then we can set up the task list as follows:

```
int j=0;
int task_B_Arg;

TaskList[j].ftpr = task_A();
TaskList[j].arg_ptr = NULL;
TaskList[j].state = STATE_INACTIVE;
TaskList[j].delay = -1;
j++;
```

```
TaskList[j].ftpr = task_B();  
task_B_Arg = 56;          // some arbitrary value  
int *ip = &task_B_Arg;  
TaskList[j].arg_ptr = (void*)ip;  
TaskList[j].state = STATE_READY;  
TaskList[j].delay = -1;  
j++;  
  
TaskList[j].fptr = NULL;  
  
... etc ...
```

In Class Exercise: Starting, Stopping, Delay

Write pseudocode to start a task, stop a task, and delay a task.

Assume `t_curr` is an integer which points to the currently running task.

```
Task_Start(int task_id) {  
    TaskList[task_id].state = STATE_READY;  
}  
  
halt_me() {  
    TaskList[t_curr].state = STATE_INACTIVE;  
}  
  
delay(int d) {  
    TaskList[t_curr].delay = d;  
    TaskList[t_curr].state = STATE_WAITING;  
}
```